STAT 405 Final Project

Alec Neme (84761030), Mikayla Liang (87701603)

Abstract

Investor confidence is fundamentally tied to market stability, as sudden transitions into high-volatility regimes expose investors to significant risk. This study explores the use of Bayesian inference methods in detecting and predicting these volatile periods. Using historical data from Yahoo Finance spanning January 1998 to the present, the comparative models identify latent shifts in market behaviour driven by rapid economic change. [Insert results].

Introduction

Stock market volatility poses significant challenges for investors, particularly during periods of economic instability. Accurately detecting these periods is critical as it allows investors to anticipate risk and make confident investment decisions. Historically, forecasting has relied on frequentist methodologies, such as ARIMA and GARCH (Patel, 2024). However, these models are limited by their inability to incorporate prior context or macroeconomic indicators. In contrast, Bayesian approaches integrate prior probability distributions and typically achieve greater accuracy in predicting volatility.

To determine how to best capture latent shifts in the market, this study compares two Bayesian methods: a standard Hidden Markov Model (HMM) and a Hierarchical Hidden Markov Model (HHMM). The HMM models the market as switching between two latent states to capture sudden shifts in market behaviour. The HHMM extends this by introducing a hierarchical structure to the emission variances. Instead of relying on fixed priors that are highly sensitive to the scale of log returns, the HHMM uses a hyperparameter to determine the rate of volatility. This flexibility allows the model to better accommodate the "fat tails" in financial data, which prevents single-week outliers from impacting the latent state estimation and producing more accurate regime probabilities. Both models are fitted using two inferential methods: Hamiltonian Monte Carlo (HMC), which produces an accurate posterior by sampling, and Variational Inference (VI), which approximates the posterior through optimization. By applying both methods to each model, we can evaluate the tradeoff between accuracy and computational efficiency.

This study spans three major stock market crises: the Dot-com Bubble, the 2008 Financial Crisis, and the 2020 COVID-19 crash. The S&P 500 Index (ˆGSPC) serves as the primary dataset for this study, as it is widely regarded as the most complete benchmark of U.S. equity market performance. Given that these crises were predominantly driven by U.S. market dynamics, the S&P 500 accurately reflects the desired market volatility. Additional exploration of other tickers is contained int the Appendix.

Literature Review

Hidden Markov Models are commonly applied to financial time series analysis and regime detection. Rabiner (1989) established the foundational algorithmic framework for HMMs, introducing the Forward-Backward algorithm that underpin modern HMM implementations. This study utilizes the Forward Algorithm introduced in Rabiner's dynamic programming formulation. Hamilton (1989) demonstrated how regime-switching models can be applied to macroeconomic time series, and showed that non-stationary economic data can

be effectively modelled as transitions between discrete latent states. These works provided the theoretical motivation for applying Bayesian models to financial markets.

For frequentist volatility modelling, Engle (1982) introduced the ARCH model and its GARCH extension, which remain the industry standard for capturing volatility clustering in financial returns. While GARCH models are effective at modelling time series volatility, they are inable to incorporate prior knowledge or quantify uncertainty in a principled Bayesian framework. This limitation motivates our use of HMMs with Bayesian inference instead.

Wang et al. (2020) represent the most closely related work to this study, applying a Gaussian HMM directly to S&P 500 data to identify market regimes using a frequentist Baum-Welch expectation-maximization algorithm. This study fits the same model class using Bayesian inference instead, generating uncertainty quantification and the incorporation of prior knowledge. We extend this framework to a Hierarchical HMM and address the prior sensitivity limitations present in the standard model.

The remainder of this study is structured as follows. Section I describes the data collection process, and defines the two Bayesian models and inferential methods used in the analysis. Section III presents the implementation and results of each model, including parameter estimates and regime probability visualizations. Section IV discusses the comparative findings, and Section V states the conclusions of the study.

Section I: Methodology

Research Objectives

The key objective of this study is to investigate the following: between discrete regime shifts (HMM) and continuous volatility evolution (Stochastic Volatility), which framework better explains the market dynamics during the Dot-com bubble, the 2008 Financial Crisis and the COVID-19 crisis? 2. To reduce the computational burden while preserving essential volatility clustering, the daily S&P500 data will be mutated to only include the final trading day of each week to establish consistent weekly boundaries. These weekly prices will then be transformed into weekly log returns, calculated as $Y_t = \ln(P_t/P_{t-1})$. This transformation is critical, as it ensures a stationary distribution of the time series which facilitates its modelling.

Hidden Markov Model (Model I)

The Likelihood

Let Y_t represent the weekly log return of the index at time t, and let $X_t \in \{1, 2\}$. State 1 represents a "Calm/Normal" market regime, and State 2 represents a "Crisis" regime.

$$Y_t \mid X_t = i \sim N(0, \sigma_i^2)$$

Where:

$$\sigma_1 \sim Exponential(1)$$

$$\sigma_{diff} \sim Exponential(1)$$

$$\sigma_2 = \sigma_1 + \sigma_{diff}$$

The Latent State Dynamics (The Regime Switching)

The sequence of unobserved market states follows a first-order Markov process. The probability distribution of tomorrow's state, X_t, is explicitly conditioned on today's state, X_{t-1}.

Latent States

$$X_t \mid X_{t-1} = 1 \sim \text{Categorical}(P_{11}, 1 - P_{11})$$
$$X_t \mid X_{t-1} = 2 \sim \text{Categorical}(1 - P_{22}, P_{22})$$

Transition Probabilities

$$P = \begin{pmatrix} P_{11} & 1 - P_{11} \\ 1 - P_{22} & P_{22} \end{pmatrix}$$

where,

$$P_{11} \sim \text{Beta}(1, 1)$$
$$P_{22} \sim \text{Beta}(1, 1)$$

We chose uniformative priors to keep the model more simple.

Hierarchical Hidden Markov Model (Model II)

While our baseline HMM successfully captures discrete regime shifts, relying on fixed parameters (such as $\sigma_i \sim \text{Exponential}(1)$) introduces a vulnerability: the posterior may be overly sensitive to these rigid prior assumptions. To mitigate this and capture the true scale of market volatility driven by the data, we introduce a Hierarchical Hidden Markov Model (HHMM).

Because weekly log returns are practically indistinguishable from zero and a "crisis" is defined primarily by an explosion in variance rather than a shift in the mean, we simplify the emission model by assuming a constant global mean of 0 across both states, same as in our simple model. This isolates the regime-switching mechanics entirely on the market volatility.

Hierarchical Parameter Definitions

The observed weekly log returns are modeled with a mean of 0 and state-dependent hierarchical variances:

$$Y_t \mid X_t = i \sim N(0, \sigma_i^2)$$

Rather than fixing the rate of the Exponential prior, we introduce a hyperparameter λ drawn from a Gamma distribution:

$$\lambda \sim \text{Gamma}(2, 0.1)$$
$$\sigma_1 \sim \text{Exponential}(\lambda)$$
$$\sigma_2 \sim \text{Exponential}(\lambda) \quad \text{subject to } \sigma_2 > \sigma_1$$

$$P_{11} \sim \text{Beta}(50, 2)$$

$$P_{22} \sim \text{Beta}(20, 2)$$

We chose these beta priors because we want market regimes to be sticky (hard to transition from calm to crisis). The crisis should be less persistent, because the world actively works to come out of a crisis. The Gamma distribution was chosen for λ since it is the conjugate prior of the Exponential Distribution.

Hamiltonian Monte Carlo (HMC)

Because Stan's Hamiltonian Monte Carlo (HMC) algorithm relies on continuous gradient calculations, it cannot directly sample discrete latent parameters. Consequently, computing the likelihood requires summing over the two states $\{1, 2\}$ for all T weeks:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \sum_{x_2=1}^{2} \cdots \sum_{x_T=1}^{2} p(Y_{1:T}, X_{1:T} = x_{1:T}|\theta, P)$$

The fully expanded equation involves exhaustive summations that requires computing the $O(2^T)$ sum at every MCMC step.

Given this approach is computationally expensive, we utilized Rao-Blackwellization Forward Algorithm to sum out the discrete state sequence $X_{1:T}$. The marginalization is computationally feasible through the Forward Algorithm, a method which exploits the Markov property to compute the marginal likelihood recursively in a single pass through the data. This method reduces the complexity from O(2T){O}(2ˆT) to {O}(T). A full derivation of the Forward Algorithm and its log-space implementation is provided in the Appendix.

Therefore the final marginalized log likelihood as computed as follows:

$$\log L(\theta, P; Y_{1:T}) = \log \left(\sum_{j=1}^{2} \exp(\log \alpha_T(j)) \right)$$

All computations are performed in log-space using Stan's numerically stable `log_sum_exp` function to prevent floating-point underflow.

Variational Inference (VI)

We additionally fit both HMM and Hierarchical HMM models using Variational Inference (VI). This approach is intended to evaluate the tradeoff between inferential accuracy and computational efficiency in our methods. Rather than sampling from the true posterior like HMC, VI reframes inference as an optimization problem. Specifically, we seek the distribution $q_\phi^* \in Q$ that most closely approximates the true posterior $\pi(x)$, as measured by the KL divergence:

$$q_\phi^* = \arg\min_\phi \mathrm{KL}(q_\phi \| \pi)$$

We adopt the meanfield variational family, which approximates the posterior using a multivariate normal distribution with a diagonal covariance matrix. The design of our Stan model $\sigma_2 = \sigma_1 + \sigma_{diff}$ encodes the primary parameter relationships directly, making the meanfield independence assumption reasonable for our model.

Results & Discussion

We started by just using the forward algorithm to calculate "filtered" probabilities. Because this method only looks at past and present data, it was way too reactive; the model basically panicked at every little market dip, giving us very jittery results instead of identifying actual, long-term crisis periods. To smooth out the jitter, we added the backward algorithm to calculate "smoothed" probabilities, drawing on concepts from Rabiner (1989) and a 2020 MDPI paper, which is explained in detail in the appendix. The goal here was to let the model look at the entire timeline of data at once, and we hoped it would realize that crashes

are sustained events. Other potential limitations of the model, was that prior sensitivity may be a thing, especially for market volatility. We would generally expect the states of the markets to be persistent, meaning parameters like P_{11} and P_{22} should have higher values. Because of this, using the rigid exponential(1) prior for the market volatilities, and the Beta(1, 1) prior for transition probabilities may be actively restricting the model. For the Stan implementation of both models, please refer to the appendix.

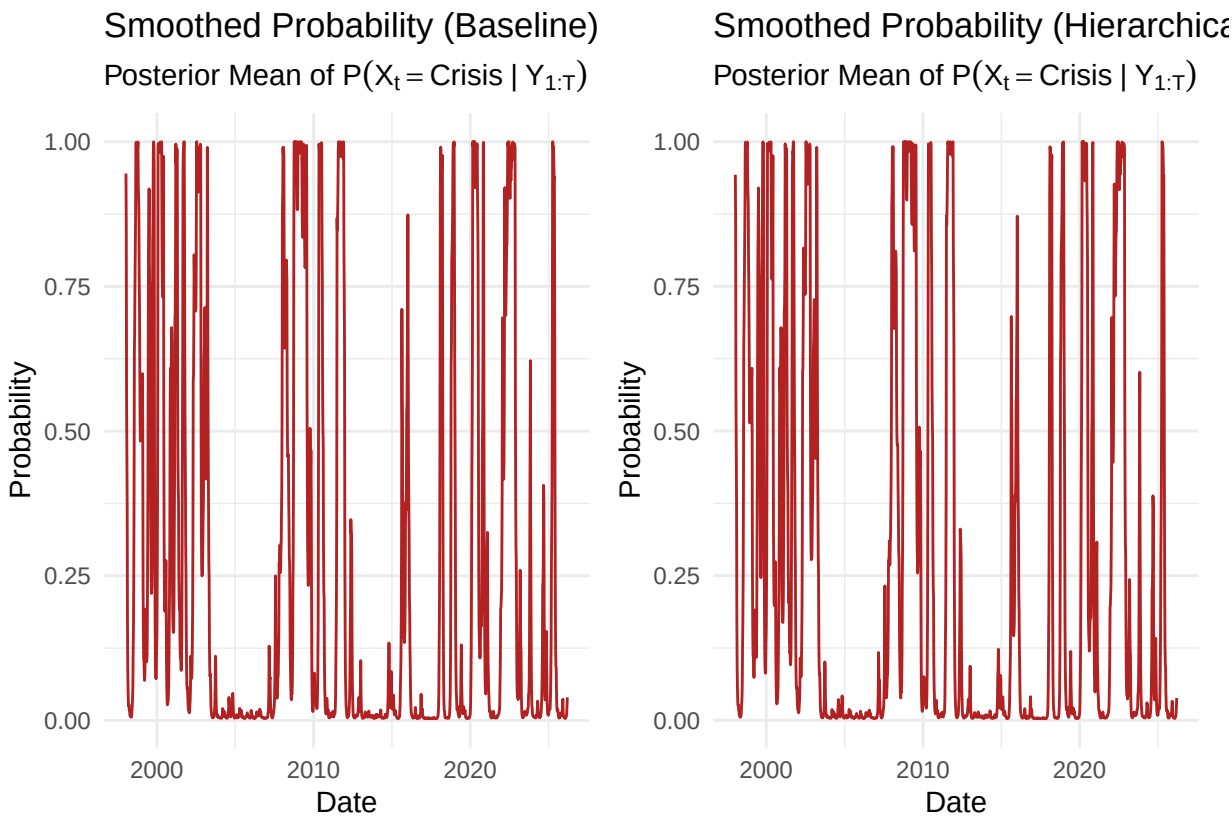


Figure 1: Figure 1.1: Smoothed Probability of Market Crisis Regimes

Both the baseline and hierarchical Hidden Markov Models successfully identified two distinct market environments within the S&P 500 log returns, demonstrating remarkable parameter robustness across both (full parameter point estimates and 95% credible intervals for all models are provided in the Appendix). In both models, the normal growth state (State 1) is characterized by a low weekly volatility of approximately 1.62% and exhibits extreme persistence, with a 97.0% to 97.2% probability (P_{11}) of remaining in this stable environment week-over-week. Conversely, the crisis regime (State 2) shows a massive volatility jump to approximately 4.08% to 4.11% per week, which is roughly 2.5 times higher than the baseline. This crisis state has a slightly lower persistence (P_{22} ranging from 0.909 to 0.916), reflecting the economic reality that while financial panics are sustained, intensely volatile events, they typically resolve faster than prolonged periods of normal market growth. The smoothed probability plot for the hierarchical model confirms these findings by clearly pinning the probability of a crisis at 1.00 during major economic shocks like the 2008 financial crisis and the 2020 COVID-19 crash. However, the jittery behavior still continues, which further highlights the binary model's tendency to switch rapidly when attempting to classify more moderate periods of market turbulence. Despite the hierarchical model introducing a more complex prior structure, both models converged on essentially the same baseline and crisis volatility scales. This striking similarity suggests that the likelihood signal from the S&P 500 data is exceptionally strong, effectively overwhelming the structural differences between the flat and hierarchical priors.

In figure 1.1, we can observe that both models successfully capture broad market regimes. However, its

reliance on a strict binary state space ($X_t \in \{1, 2\}$) introduces structural limitations. In our current architecture, the model is forced into an "all-or-nothing" problem, attempting to classify every trading week as either perfectly normal or a total crisis. Because the transition waiting times are geometrically distributed, the model assumes the probability of exiting a market regime decays rigidly over time. This results in our model oscillating between the crisis and calm state. A solution to this problem would be expanding the latent state space to a four-state model ($X_t \in \{1, 2, 3, 4\}$) with explicit macroeconomic interpretations: Severe Crisis, Mild Crisis (or market correction), Normal Growth, and Extreme Growth (or bull run). This expansion allows the model to capture a more natural life cycle of a financial regime, instead of having a black or white rigid approach to categorizing. For example, a market could transition from Extreme Growth into a Mild Crisis before cascading into a Severe Crisis. Most importantly, this multi-state approach could possibly resolve the geometric waiting time limitation for broader economic cycles.

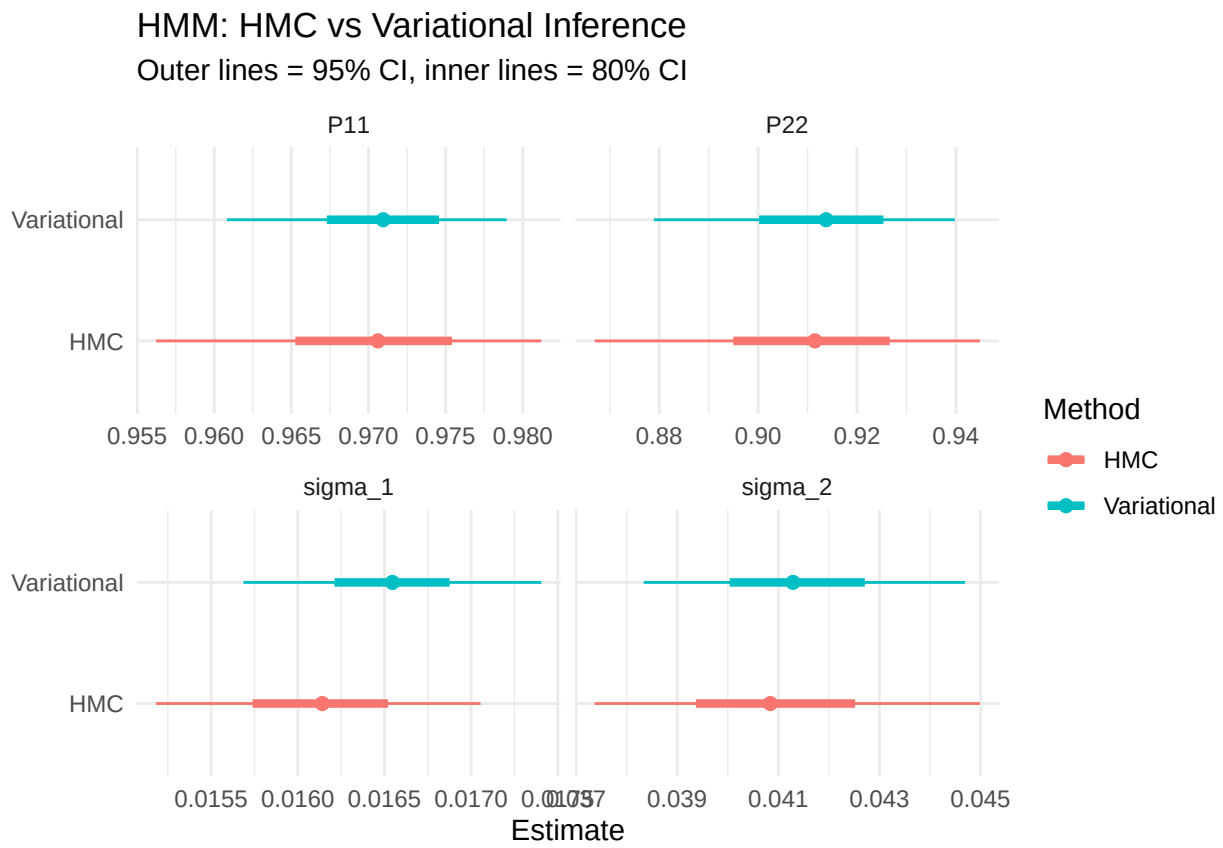
Correctness Checks, Goodness of Fit, and MCMC Diagnostics

To validate the computational correctness of our models and rule out software defects, we performed a 100-run Simulation-Based Calibration (SBC) using synthetic datasets matching the full length of our sample. For a 90% credible interval, a properly calibrated model should capture the true parameters approximately 90% of the time. For our baseline model, the empirical coverage results were 0.91 for P_{11}, 0.92 for P_{22}, 0.87 for σ_1, and 0.89 for σ_2. The hierarchical model yielded similarly robust results, with coverages of 0.88 for P_{11}, 0.89 for P_{22}, 0.87 for σ_1, and 0.89 for σ_2. While some parameters sit slightly below the nominal 0.90 target, this deviation is entirely expected due to Monte Carlo error. With 100 simulations, the binomial standard deviation for a 90% probability is roughly 3%, meaning any result between 0.87 and 0.93 falls within normal statistical noise. Because the coverage for all parameters across both models falls strictly within this expected range, we can conclude that both Stan implementations are mathematically sound and defect-free.

To evaluate the goodness of fit and out-of-sample predictive calibration of both models, we performed a rolling Leave-One-Out (LOO) cross-validation over a 50-week period, chosen to keep computational times manageable. To preserve the sequential dependence of the Hidden Markov Models, we iteratively truncated the time series at a moving boundary t, requiring the models to generate a one-step-ahead forecast for week $t + 1$. For each of the 50 iterations, we computed the 90% posterior predictive credible interval for the unobserved log return. The baseline model yielded a coverage proportion of 0.94 (capturing 47 of the 50 out-of-sample weeks). The hierarchical model yielded an empirical coverage proportion of 0.92 (capturing 46 of the 50 weeks). This result indicates exceptional model calibration, matching the nominal 90% target even more closely than the baseline model without being overly conservative. This confirmed that both models do not suffer from gross mis-specification and provide reliable bounds for near-term forecasting.

The sampler converged well across all models, exhibiting no torpid mixing. Specifically, all four MCMC chains were stationary and thoroughly overlapping, with rank histograms displaying a uniform distribution. This confirms that the sampler efficiently and reliably explored the parameter space for both the baseline and hierarchical models. Full MCMC diagnostic plots, including trace plots and rank histograms, are provided in the Appendix.

Variational Inference



Across both the HMM and Hierarchical HMM, VI produced similar point estimates and smoothed probabilities (visualized in the Appendix) compared to HMC, but differed noticeably in how they measured uncertainty and in their runtimes. HMC took roughly 33 seconds for both models, resulting in well-calibrated credible intervals that accurately reflect the full uncertainty of the posterior. On the other hand, VI converged in just 5.9 seconds for both models, averaging 5.7× faster than HMC. However, this speed was achieved at the cost of artificially narrow, under-dispersed credible intervals as seen above in Figure 2. This under-estimation of uncertainty is a well-known consequence of the mean-field independence assumption, which ignores posterior correlations between parameters. Overall, the pattern was consistent: HMC gives accurate, reliable uncertainty estimates but takes longer, while VI is much faster but sacrifices inferential precision by under-reporting variance.

References

Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. Econometrica, 57(2), 357–384. https://doi.org/10.2307/1912559

Patel, R. M. (2024). Bayesian estimation techniques in stock market forecasting: An empirical analysis using Wipro stock data. International Journal for Innovative Research in Multidisciplinary Field, 10(10), 35–38. https://www.ijirmf.com/wp-content/uploads/IJIRMF202410007-min.pdf

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2), 257–286. https://doi.org/10.1109/5.18626

V-Lab. (n.d.). GARCH model documentation. NYU Stern Volatility Institute. https://vlab.stern.nyu.edu/docs/volatility/GARCH

Wang, M., Lin, Y.-H., & Mikhelson, I. (2020). Regime-switching factor investing with hidden Markov models. Journal of Risk and Financial Management, 13(12), 311. https://doi.org/10.3390/jrfm13120311

Appendix

To validate the accuracy of our approach, we may utilize the CBOE Volatility Index (VIX), allowing us to compare our model's estimated latent volatility against Wall Street's implied volatility. Additionally, assets such as sector-specific ETFs (XLK, XLF) and a safe-haven treasury bond (TLT) will be incorporated to test if the model's regime-switching logic generalizes reliably across different market segments, as well as sector-specific insight of each crisis. The relevant ETFs are defined below:

- ^GSPC (S&P 500 Index): Top 500 stocks in the U.S. This is the observation that we will most likely use.
- ^VIX (CBOE Volatility Index): Market volatility measure that we can use for model validation.
- XLK (Technology Select Sector SPDR Fund): An ETF containing all the tech stocks in the S&P 500.
- XLF (Financial Select Sector SPDR Fund): An ETF containing all the financial stocks in the S&P 500 (banks, insurance, etc.).
- TLT (iShares 20+ Year Treasury Bond ETF): This tracks long-term US government bonds. "Safe haven" asset that normally behaves opposite to stocks.

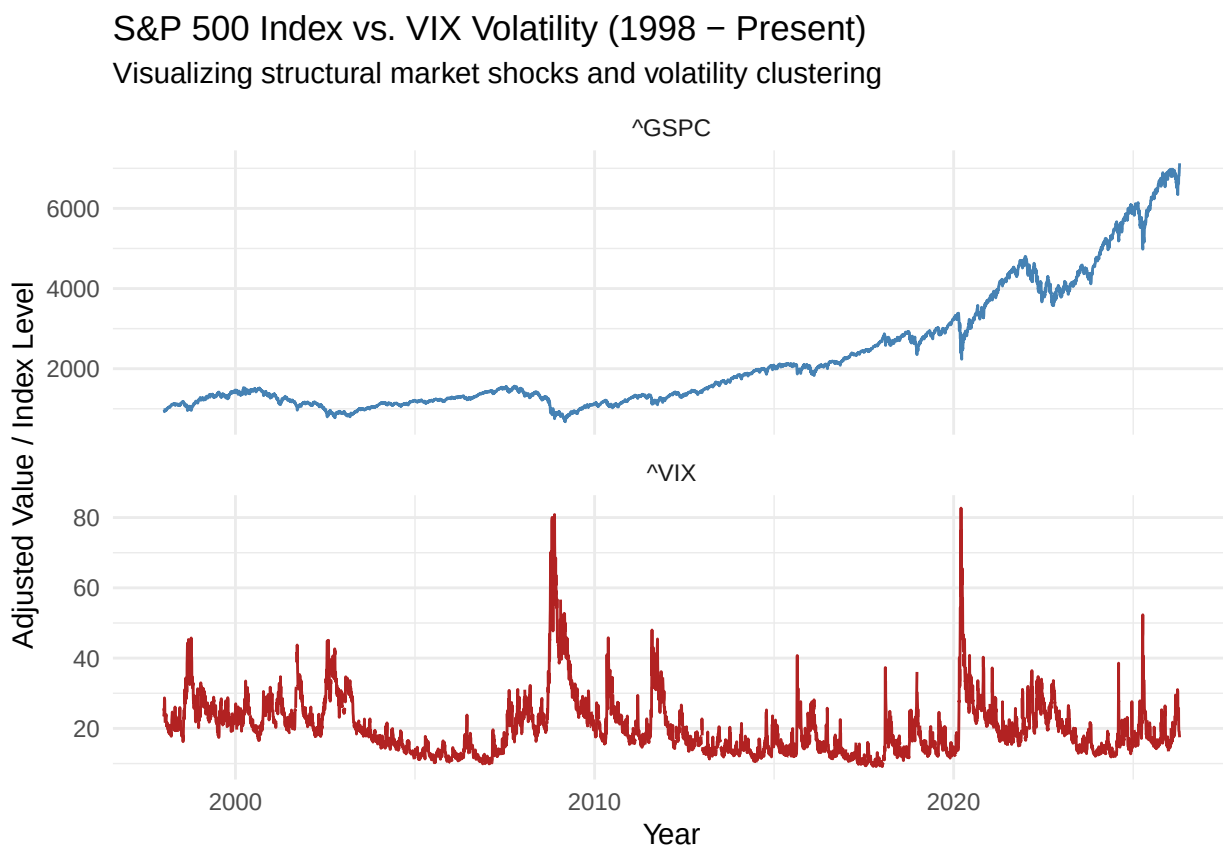
EDA

```r
project_tickers <- c("^GSPC", "^VIX", "XLK", "XLF", "TLT")
market_data <- tq_get(project_tickers,
                      get  = "stock.prices",
                      from = "1998-01-01")

weekly_returns <- market_data %>%
  mutate(week = floor_date(date, "week")) %>%
  group_by(symbol, week) %>%
  arrange(date) %>%
  slice_tail(n = 1) %>%
  ungroup()


weekly_log_returns <- weekly_returns %>%
  group_by(symbol) %>%
  arrange(date) %>%
  mutate(
    previous_price = lag(adjusted),
    log_return = log(adjusted / previous_price)
  ) %>%
  filter(!is.na(log_return)) %>%
  select(symbol, date, adjusted, log_return)

head(weekly_log_returns)
```

```
## # A tibble: 6 x 4
## # Groups:   symbol [2]
##   symbol date       adjusted log_return
##   <chr>  <date>        <dbl>      <dbl>
## 1 ^GSPC  1998-01-09    928.    -0.0498
```

```
## 2 ^VIX   1998-01-09     28.7    0.203
## 3 ^GSPC  1998-01-16    962.     0.0358
## 4 ^VIX   1998-01-16     21.6   -0.282
## 5 ^GSPC  1998-01-23    958.    -0.00409
## 6 ^VIX   1998-01-23     23.3    0.0743
```

```r
market_data %>%
  filter(symbol %in% c("^GSPC", "^VIX")) %>%
  ggplot(aes(x = date, y = adjusted, color = symbol)) +
  geom_line(show.legend = FALSE) +
  facet_wrap(~symbol, scales = "free_y", ncol = 1) +
  theme_minimal() +
  labs(title = "S&P 500 Index vs. VIX Volatility (1998 - Present)",
       subtitle = "Visualizing structural market shocks and volatility clustering",
       x = "Year",
       y = "Adjusted Value / Index Level") +
  scale_color_manual(values = c("^GSPC" = "steelblue", "^VIX" = "firebrick"))
```



The visualization highlights a clear inverse correlation: significant price contractions in the S&P 500 (^GSPC) correspond directly to sharp spikes in the VIX. These synchronized movements clearly mark the three major structural shocks identified in our proposal. The abrupt, vertical nature of these volatility surges visually mimics discrete state transitions, reinforcing our decision to utilize a Hidden Markov Model (HMM).

Furthermore, the raw S&P 500 series exhibits a distinct upward trend over the 25-year period, indicating it is non-stationary. By transforming these prices into weekly log returns, $Y_t = \ln(P_t/P_{t-1})$, we remove this drift and stabilize the variance. This transformation is mathematically critical; it ensures our HMM operates

on a stationary distribution, which is a necessary condition for the model to reliably infer the "Calm" and "Crisis" latent states over the long term.

Rao-Blackwellization via the Forward Algorithm

Over the whole dataset, the brute-force marginalized joint likelihood requires summing over the two states $\{1, 2\}$ for all T weeks:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \sum_{x_2=1}^{2} \cdots \sum_{x_T=1}^{2} p(Y_{1:T}, X_{1:T} = x_{1:T}|\theta, P)$$

By factoring the joint distribution into the initial state probabilities π, the transition matrix P, and the state-dependent emission likelihoods, we obtain the fully expanded equation:

$$p(Y_{1:T}|\theta, P) = \sum_{x_1=1}^{2} \cdots \sum_{x_T=1}^{2} \left[\pi_{x_1} p(Y_1|\mu_{x_1}, \sigma^2_{x_1}) \prod_{t=2}^{T} P_{x_{t-1},x_t} p(Y_t|\mu_{x_t}, \sigma^2_{x_t}) \right]$$

This exhaustive summation involves 2^T unique possible paths. Computing this $O(2^T)$ sum at every MCMC step is computationally expensive.

To resolve this computational bottleneck, we utilize the Forward Algorithm. The Forward Algorithm is a dynamic programming approach that relies on the conditional independence of the HMM. Instead of calculating the probability of every single distinct path from week 1 to week T independently, the algorithm calculates the probability sequentially and caches the intermediate results.

Because the Markov property dictates that the future state depends only on the present state, we do not need to track how the market arrived at a certain state, only the accumulated probability mass of being there. We define a forward variable, $\alpha_t(j)$, representing the joint probability of observing the sequence of returns up to time t and currently being in state j:

$$\alpha_t(j) = p(Y_{1:t}, X_t = j|\theta, P)$$

The algorithm computes this dynamically in three steps:

1. Base Case ($t = 1$):

$$\alpha_1(j) = \pi_j p(Y_1|0, \sigma^2_j) \quad \text{for } j \in \{1, 2\}$$

2. Recursive Step ($t = 2, \ldots, T$): By summing the cached probabilities from the previous week ($\alpha_{t-1}(i)$) and multiplying by the transition probability P_{ij}, we find the total probability of arriving at state j. We then multiply by the new emission likelihood:

$$\alpha_t(j) = p(Y_t|0, \sigma^2_j) \sum_{i=1}^{2} \alpha_{t-1}(i) P_{ij}$$

3. Termination (Marginalized Likelihood): At the final week T, the algorithm has compressed the 2^T paths into just two values: $\alpha_T(1)$ and $\alpha_T(2)$, representing the joint probability of the entire dataset ending in State 1 and State 2, respectively. By the Law of Total Probability, since the market must end in one of these two mutually exclusive states, the final marginalized likelihood is simply their sum:

$$L(\theta, P; Y_{1:T}) = p(Y_{1:T}|\theta, P) = \sum_{j=1}^{2} \alpha_T(j)$$

This reduces the time complexity from $O(2^T)$ to $O(T)$.

Log-Space Implementation

While the Forward Algorithm solves the theoretical complexity, implementing it directly may cause underflow. Multiplying hundreds of small probability values quickly results in a number smaller than a computer's floating-point limit (e.g., 10^{-308}), causing the software to round the total likelihood to zero.

To maintain numerical stability, Stan evaluates the marginalized posterior strictly in log-space. We define the log-forward variable as $\log \alpha_t(j)$.The base case becomes a simple addition:

$$\log \alpha_1(j) = \log \pi_j + \log p(Y_1|0, \sigma_j^2)$$

The recursive step requires taking the log of a sum, which Stan handles using the stable log_sum_exp function:

$$\log \alpha_t(j) = \log p(Y_t|0, \sigma_j^2) + \log \left(\sum_{i=1}^{2} \exp(\log \alpha_{t-1}(i) + \log P_{ij}) \right)$$

Finally, the total marginalized log-likelihood is the log-sum-exp of the final terminal states:

$$\log L(\theta, P; Y_{1:T}) = \log \left(\sum_{j=1}^{2} \exp(\log \alpha_T(j)) \right)$$

The Backward Algorithm and Smoothed Probabilities

While the Forward algorithm successfully computes the marginalized likelihood required to fit the model, the latent state probabilities it generates are "filtered." The filtered probability, $P(X_t = j \mid Y_{1:t})$, estimates the current market regime using only the data available up to time t. Because financial markets are inherently noisy, relying solely on past and present data makes the model hyper-reactive. A single week of extreme volatility might cause the filtered probability to spike into a "Crisis" state, only to immediately revert the following week, failing to capture the sustained nature of true macroeconomic shifts.

To resolve this and accurately identify prolonged market regimes, we must evaluate the probability of the latent state given the *entire* sequence of observations, $P(X_t = j \mid Y_{1:T})$. This is known as the "smoothed" probability. To compute this, we introduce the Backward algorithm.

Much like the Forward algorithm, the Backward algorithm is a dynamic programming approach, but it operates in reverse. We define the backward variable, $\beta_t(j)$, as the probability of observing all future data from $t+1$ to T, given that the market is currently in state j at time t:

$$\beta_t(j) = p(Y_{t+1:T} \mid X_t = j, \theta, P)$$

The algorithm computes this recursively starting from the end of the time series:

1. Base Case ($t = T$): We initialize the final backward variables to 1 (or 0 in log-space), as there are no future observations beyond T.

$$\beta_T(j) = 1 \quad \text{for } j \in \{1, 2\}$$

2. Recursive Step ($t = T-1, T-2, \ldots, 1$): To compute the backward variable for week t, we sum over all possible states i for the next week, multiplying the transition probability P_{ji}, the emission likelihood of tomorrow's observation, and tomorrow's cached backward variable:

$$\beta_t(j) = \sum_{i=1}^{2} P_{ji} p(Y_{t+1} \mid \mu_i, \sigma_i^2) \beta_{t+1}(i)$$

3. Computing the Smoothed Probabilities (γ): Once both the forward (α) and backward (β) passes are complete, we combine them. By Bayes' Theorem and the conditional independence properties of the HMM, the joint probability of the entire dataset and being in state j at time t is simply the product of the two variables. We normalize this product to obtain $\gamma_t(j)$, the final smoothed probability:

$$\gamma_t(j) = P(X_t = j \mid Y_{1:T}) = \frac{\alpha_t(j)\beta_t(j)}{\sum_{i=1}^{2} \alpha_t(i)\beta_t(i)}$$

By incorporating the backward pass, the model gains the benefit of hindsight. It can look ahead at the future trajectory of the S&P 500 to determine whether a sudden drop was a brief, one-week anomaly or the beginning of a sustained structural crash like the 2008 Financial Crisis.

Stan implementation HMM

```
data {
  int<lower=1> T;
  vector[T] Y;
}

parameters {
  real<lower=0> sigma_1;
  real<lower=0> sigma_diff;
  real<lower=0, upper=1> P11;
  real<lower=0, upper=1> P22;
}

transformed parameters {
  real<lower=0> sigma_2;
  array[2] real<lower=0> sigma;
  matrix[2, 2] log_P;
  matrix[T, 2] log_alpha_full;

  sigma_2 = sigma_1 + sigma_diff;

  sigma[1] = sigma_1;
  sigma[2] = sigma_2;

  log_P[1, 1] = log(P11);
  log_P[1, 2] = log1m(P11);
  log_P[2, 1] = log1m(P22);
  log_P[2, 2] = log(P22);

  for (j in 1:2)
    log_alpha_full[1, j] = log(0.5) + normal_lpdf(Y[1] | 0, sigma[j]);
```

```
  for (t in 2:T) {
    for (j in 1:2) {
      vector[2] accumulator;
      for (i in 1:2)
        accumulator[i] = log_alpha_full[t-1, i] + log_P[i, j];
      log_alpha_full[t, j] = normal_lpdf(Y[t] | 0, sigma[j]) + log_sum_exp(accumulator);
    }
  }
}

model {
  sigma_1 ~ exponential(1);
  sigma_diff ~ exponential(1);

  P11 ~ beta(1, 1);
  P22 ~ beta(1, 1);

  target += log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]);
}

generated quantities {
  matrix[T, 2] log_beta;
  matrix[T, 2] log_gamma;
  vector[T] prob_crisis;

  real y_predict;
  real p_state1_T;
  real p_state2_T;
  real p_state1_next;
  int predicted_state;

  for (j in 1:2) {
    log_beta[T, j] = 0.0;
  }

  for (t_rev in 1:(T - 1)) {
    int t = T - t_rev;

    for (i in 1:2) {
      vector[2] accumulator;
      for (j in 1:2) {
        accumulator[j] = log_P[i, j] + normal_lpdf(Y[t+1] | 0, sigma[j]) + log_beta[t+1, j];
      }
      log_beta[t, i] = log_sum_exp(accumulator);
    }
  }

  for (t in 1:T) {
    log_gamma[t, 1] = log_alpha_full[t, 1] + log_beta[t, 1];
    log_gamma[t, 2] = log_alpha_full[t, 2] + log_beta[t, 2];

    prob_crisis[t] = exp(log_gamma[t, 2] - log_sum_exp(log_gamma[t, 1], log_gamma[t, 2]));
  }
```

```
  p_state1_T = exp(log_alpha_full[T, 1] - log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]));
  p_state2_T = exp(log_alpha_full[T, 2] - log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]));

  p_state1_next = p_state1_T * P11 + p_state2_T * (1 - P22);

  predicted_state = bernoulli_rng(1 - p_state1_next) + 1;

  y_predict = normal_rng(0, sigma[predicted_state]);
}
```

Stan implementation HHMM

```
data {
  int<lower=1> T;
  vector[T] Y;
}

parameters {
  real<lower=0> lambda;
  real<lower=0> sigma_1;
  real<lower=0> sigma_diff;
  real<lower=0, upper=1> P11;
  real<lower=0, upper=1> P22;
}

transformed parameters {
  real<lower=0> sigma_2;
  array[2] real<lower=0> sigma;
  matrix[2, 2] log_P;
  matrix[T, 2] log_alpha_full;

  sigma_2 = sigma_1 + sigma_diff;

  sigma[1] = sigma_1;
  sigma[2] = sigma_2;

  log_P[1, 1] = log(P11);
  log_P[1, 2] = log1m(P11);
  log_P[2, 1] = log1m(P22);
  log_P[2, 2] = log(P22);

  for (j in 1:2)
    log_alpha_full[1, j] = log(0.5) + normal_lpdf(Y[1] | 0, sigma[j]);

  for (t in 2:T) {
    for (j in 1:2) {
      vector[2] accumulator;
      for (i in 1:2)
        accumulator[i] = log_alpha_full[t-1, i] + log_P[i, j];
      log_alpha_full[t, j] = normal_lpdf(Y[t] | 0, sigma[j]) + log_sum_exp(accumulator);
    }
```

```
  }
}

model {
  lambda ~ gamma(2, 0.1);
  sigma_1 ~ exponential(lambda);
  sigma_diff ~ exponential(lambda);

  P11 ~ beta(50, 2);
  P22 ~ beta(20, 2);

  target += log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]);
}

generated quantities {
  matrix[T, 2] log_beta;
  matrix[T, 2] log_gamma;
  vector[T] prob_crisis;

  real y_predict;
  real p_state1_T;
  real p_state2_T;
  real p_state1_next;
  int predicted_state;

  for (j in 1:2) {
    log_beta[T, j] = 0.0;
  }

  for (t_rev in 1:(T - 1)) {
    int t = T - t_rev;

    for (i in 1:2) {
      vector[2] accumulator;
      for (j in 1:2) {
        accumulator[j] = log_P[i, j] + normal_lpdf(Y[t+1] | 0, sigma[j]) + log_beta[t+1, j];
      }
      log_beta[t, i] = log_sum_exp(accumulator);
    }
  }

  for (t in 1:T) {
    log_gamma[t, 1] = log_alpha_full[t, 1] + log_beta[t, 1];
    log_gamma[t, 2] = log_alpha_full[t, 2] + log_beta[t, 2];

    prob_crisis[t] = exp(log_gamma[t, 2] - log_sum_exp(log_gamma[t, 1], log_gamma[t, 2]));
  }

  p_state1_T = exp(log_alpha_full[T, 1] - log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]));
  p_state2_T = exp(log_alpha_full[T, 2] - log_sum_exp(log_alpha_full[T, 1], log_alpha_full[T, 2]));

  p_state1_next = p_state1_T * P11 + p_state2_T * (1 - P22);
```

```
  predicted_state = bernoulli_rng(1 - p_state1_next) + 1;

  y_predict = normal_rng(0, sigma[predicted_state]);
}
```

Hidden Code Used in Results and Discussion:

```
# model_hmm <- cmdstan_model("hmm_volatility.stan")
#
# fit_hmc_hmm <- model_hmm$sample(
#   data = stan_data,
#   chains = 4,
#   parallel_chains = 4,
#   iter_warmup = 1000,
#   iter_sampling = 1000,
#   adapt_delta = 0.95,
#   seed = 405,
#   refresh = 0,
#   show_messages = FALSE
# )
#
# model_hierarchical <- cmdstan_model("hhmm_volatility.stan")
#
# stan_data_hierarchical <- list(
#   T = nrow(returns),
#   Y = returns$log_return
# )
#
# fit_hmc_hhmm <- model_hierarchical$sample(
#   data = stan_data_hierarchical,
#   chains = 4,
#   parallel_chains = 4,
#   iter_warmup = 1000,
#   iter_sampling = 1000,
#   adapt_delta = 0.95,
#   seed = 405,
#   refresh = 0,
#   show_messages = FALSE
# )
```

```
# prob_crisis_draws <- fit_hmc_hmm$draws("prob_crisis", format = "matrix")
# mean_prob_crisis <- colMeans(prob_crisis_draws)
#
# p1 <- data.frame(
#   date = returns$date,
#   prob = mean_prob_crisis
# ) %>%
#   ggplot(aes(x = date, y = prob)) +
#   geom_line(color = "firebrick") +
#   theme_minimal() +
#   labs(
```

```
#     title = "Smoothed Probability (Baseline)",
#     subtitle = expression("Posterior Mean of " * P(X[t] == Crisis ~ "|" ~ Y["1:T"])),
#     x = "Date",
#     y = "Probability"
#   )
#
# prob_crisis_draws_heir <- fit_hmc_hhmm$draws("prob_crisis", format = "matrix")
# mean_prob_crisis_heir <- colMeans(prob_crisis_draws_heir)
#
# p2 <- data.frame(
#   date = returns$date,
#   prob = mean_prob_crisis_heir
# ) %>%
#   ggplot(aes(x = date, y = prob)) +
#   geom_line(color = "firebrick") +
#   theme_minimal() +
#   labs(
#     title = "Smoothed Probability (Hierarchical)",
#     subtitle = expression("Posterior Mean of " * P(X[t] == Crisis ~ "|" ~ Y["1:T"])),
#     x = "Date",
#     y = "Probability"
#   )
#
# p1 + p2
```

```
# fit_vi_hmm <- model_hmm$variational(
#   data = stan_data,
#   seed = 405,
#   algorithm = "meanfield",
#   output_samples = 1000,
#   refresh = 0,
#   show_messages = FALSE
# )
# fit_vi_hhmm <- model_hierarchical$variational(
#   data = stan_data_hierarchical,
#   seed = 405,
#   algorithm = "meanfield",
#   output_samples = 1000,
#   refresh = 0,
#   show_messages = FALSE
# )
```

```
# # code modified from "Advanced Inference - VI with Stan" lecture
# sel <- c("P11", "P22", "sigma_1", "sigma_2")
#
# data_hmc <- mcmc_intervals_data(fit_hmc_hmm$draws(variables = sel)) %>%
#   mutate(method = "HMC")
#
# data_vi <- mcmc_intervals_data(fit_vi_hmm$draws(variables = sel)) %>%
#   mutate(method = "Variational")
#
# bind_rows(data_hmc, data_vi) %>%
#   ggplot(aes(x = m, y = method, color = method)) +
```

```r
#   geom_linerange(aes(xmin = ll, xmax = hh),
#                  position = position_dodge(width = 0.5)) +
#   geom_linerange(aes(xmin = l, xmax = h),
#                  position = position_dodge(width = 0.5), linewidth = 1.5) +
#   geom_point(position = position_dodge(width = 0.5), size = 2) +
#   facet_wrap(~parameter, scales = "free_x", ncol = 2) +
#   theme_minimal() +
#   labs(
#     title = "HMM: HMC vs Variational Inference",
#     subtitle = "Outer lines = 95% CI, inner lines = 80% CI",
#     x = "Estimate",
#     y = NULL,
#     color = "Method"
#   )
```

Parameter results for the three models

```r
as.data.frame(fit_hmc_hmm$summary(c("P11", "P22", "sigma_1", "sigma_2")))
```

```
##   variable       mean     median           sd          mad         q5
## 1      P11 0.96996982 0.97060327 0.0077214148 0.0074812144 0.95622741
## 2      P22 0.90952238 0.91147999 0.0240743204 0.0237449361 0.86689939
## 3  sigma_1 0.01613251 0.01614101 0.0005687475 0.0005797626 0.01518233
## 4  sigma_2 0.04099485 0.04084162 0.0023261359 0.0023196693 0.03736530
##          q95      rhat ess_bulk ess_tail
## 1 0.98121107 1.0010343 2784.525 2606.269
## 2 0.94486961 1.0011561 2283.602 2521.134
## 3 0.01705436 0.9995834 2893.001 2570.527
## 4 0.04498723 1.0006070 2850.440 2684.311
```

```r
as.data.frame(fit_hmc_hhmm$summary(c("P11", "P22", "sigma_1", "sigma_2")))
```

```
##   variable       mean     median          sd          mad         q5        q95
## 1      P11 0.97152950 0.97222494 0.006971973 0.0069378119 0.95915291 0.98162235
## 2      P22 0.91609375 0.91807913 0.021193801 0.0206328327 0.87879193 0.94840415
## 3  sigma_1 0.01613030 0.01613669 0.000537351 0.0005405582 0.01524782 0.01701240
## 4  sigma_2 0.04071251 0.04059734 0.002186214 0.0021849321 0.03731597 0.04446785
##       rhat ess_bulk ess_tail
## 1 1.000279 3246.730 2638.168
## 2 1.000222 3179.009 2773.057
## 3 1.001800 3073.815 2795.086
## 4 1.001455 3021.534 2599.376
```

```r
# Print summary for VI baseline model
print(as.data.frame(fit_vi_hmm$summary(c("P11", "P22", "sigma_1", "sigma_2"))))
```

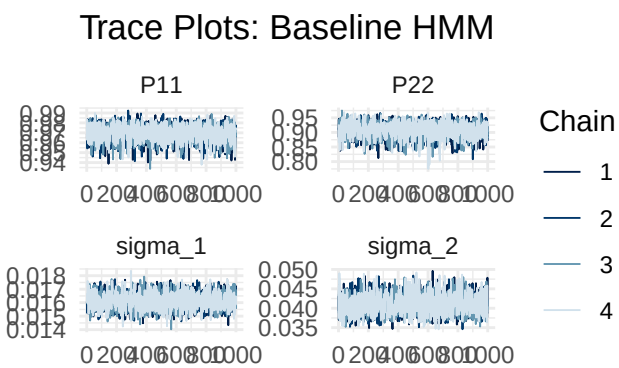
```
##   variable       mean     median           sd          mad         q5
## 1      P11 0.97062351 0.97095120 0.0055315158 0.0053849440 0.96081744
## 2      P22 0.91182232 0.91374622 0.0188570152 0.0182232889 0.87889703
## 3  sigma_1 0.01655200 0.01654607 0.0005141405 0.0004926991 0.01568665
```

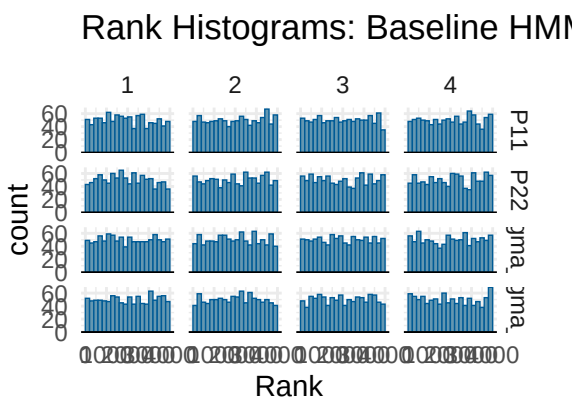
```
## 4  sigma_2 0.04138019 0.04129014 0.0019796578 0.0019746438 0.03833861
##          q95
## 1 0.97894944
## 2 0.93979863
## 3 0.01740571
## 4 0.04469357
```

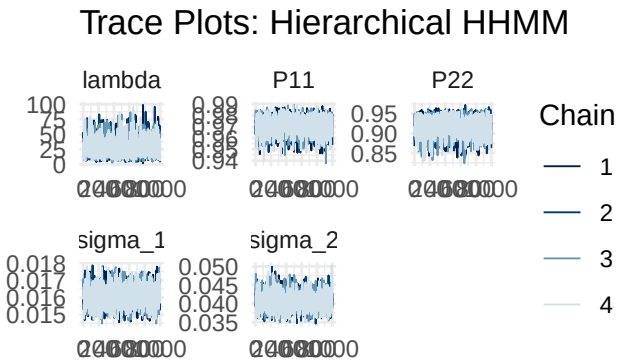
```r
# Print summary for VI hierarchical model
print(as.data.frame(fit_vi_hhmm$summary(c("P11", "P22", "sigma_1", "sigma_2"))))
```

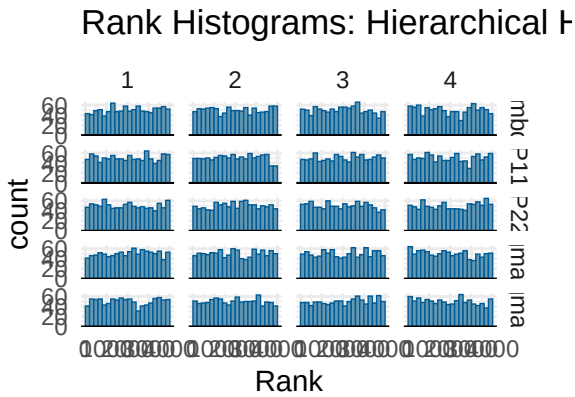
```
##   variable       mean     median           sd          mad         q5
## 1      P11 0.97330007 0.97388248 0.0066328336 0.0064791770 0.96095514
## 2      P22 0.91484033 0.91605671 0.0175209639 0.0173713425 0.88393070
## 3  sigma_1 0.01586235 0.01586420 0.0004163716 0.0004282386 0.01521999
## 4  sigma_2 0.04079202 0.04071309 0.0022948127 0.0023768354 0.03709168
##          q95
## 1 0.98294290
## 2 0.94148380
## 3 0.01655483
## 4 0.04456029
```

MCMC Diagnostic Plots









Smoothed Probability Plots for VI

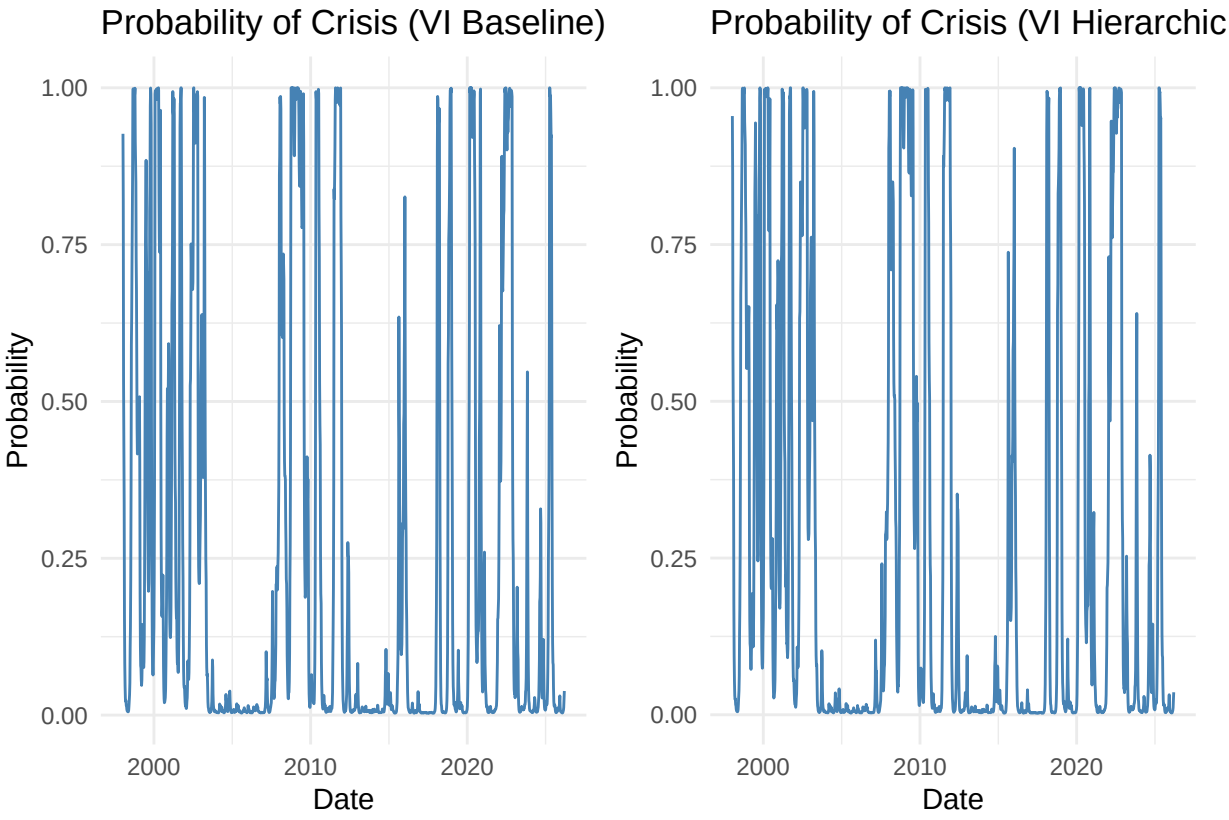


Figure 2: Smoothed Probabilities for VI approach.

Correctness Check HMM

```
# set.seed(405)
# n_sims <- 100
# T_sim <- nrow(returns)
# true_mu <- 0
# true_sigma <- c(0.015, 0.040)
# true_P <- matrix(c(0.95, 0.05, 0.10, 0.90), nrow = 2, byrow = TRUE)
#
# coverage_results <- data.frame(
#   P11_captured = logical(n_sims),
#   P22_captured = logical(n_sims),
#   sigma1_captured = logical(n_sims),
#   sigma2_captured = logical(n_sims)
# )
#
# cat("Starting 100 simulations... This will take a few minutes.\n")
#
# for (i in 1:n_sims) {
#   true_states <- numeric(T_sim)
#   true_states[1] <- 1
#   for (t in 2:T_sim) {
#     true_states[t] <- sample(1:2, size = 1, prob = true_P[true_states[t-1], ])
#   }
#
#   simulated_Y <- numeric(T_sim)
#   for (t in 1:T_sim) {
#     simulated_Y[t] <- rnorm(1, mean = true_mu, sd = true_sigma[true_states[t]])
#   }
#
#   fit_sim <- model_hmm$sample(
#     data = list(T = T_sim, Y = simulated_Y),
#     chains = 4, parallel_chains = 4, refresh = 0, show_messages = FALSE
#   )
#
#   draws <- fit_sim$draws(c("P11", "P22", "sigma_1", "sigma_2"), format = "matrix")
#   cis <- apply(draws, 2, quantile, probs = c(0.05, 0.95))
#
#   coverage_results$P11_captured[i] <- true_P[1,1] >= cis[1, "P11"] && true_P[1,1] <= cis[2, "P11"]
#   coverage_results$P22_captured[i] <- true_P[2,2] >= cis[1, "P22"] && true_P[2,2] <= cis[2, "P22"]
#   coverage_results$sigma1_captured[i] <- true_sigma[1] >= cis[1, "sigma_1"] && true_sigma[1] <= cis[2
#   coverage_results$sigma2_captured[i] <- true_sigma[2] >= cis[1, "sigma_2"] && true_sigma[2] <= cis[2
#
#   if (i %% 10 == 0) cat("Finished simulation", i, "of", n_sims, "\n")
# }
#
# cat("\n--- 100-RUN CALIBRATION RESULTS ---\n")
# print(colMeans(coverage_results))
```

LOO Goodness-of-Fit HMM

```
# subset_returns <- tail(returns, 300)
# n_total_subset <- nrow(subset_returns)
# test_weeks <- 50
#
# results_list <- list()
#
# cat("Starting 50 rolling LOO predictions...\n")
#
# for (i in test_weeks:1) {
#   current_T <- n_total_subset - i
#   actual_y <- subset_returns$log_return[current_T + 1]
#
#   # Package the truncated data up to current_T
#   stan_data_loo <- list(
#     T = current_T,
#     Y = subset_returns$log_return[1:current_T]
#   )
#
#   # Fit the model
#   fit_loop <- model_hmm$sample(
#     data = stan_data_loo,
#     chains = 4, parallel_chains = 4, refresh = 0, show_messages = FALSE
#   )
#
#   # Extract prediction and calculate 90% CI
#   y_pred <- fit_loop$draws("y_predict", format = "matrix")
#   ci <- quantile(y_pred, probs = c(0.05, 0.95))
#
#   inside <- actual_y >= ci[1] && actual_y <= ci[2]
#
#   results_list[[length(results_list) + 1]] <- data.frame(
#     Date = subset_returns$date[current_T + 1],
#     Actual = actual_y,
#     CI_Lower = ci[1],
#     CI_Upper = ci[2],
#     Captured = inside
#   )
#
#   if (i %% 10 == 0) cat("Weeks remaining to test:", i, "\n")
# }
#
# merged_loo_df <- do.call(rbind, results_list)
# proportion_covered <- mean(merged_loo_df$Captured)
#
# cat("\n--- MULTIPLE LOO GOODNESS OF FIT RESULTS ---\n")
# cat("Proportion of true values inside the 90% CI:", proportion_covered, "\n")
```

Correctness check HHMM

```
# set.seed(405)
#
# coverage_results_hierarchical <- data.frame(
#   P11_captured = logical(n_sims),
#   P22_captured = logical(n_sims),
#   sigma1_captured = logical(n_sims),
#   sigma2_captured = logical(n_sims)
# )
#
# cat("Starting 100 simulations for Hierarchical Model... This will take a few minutes.\n")
#
# for (i in 1:n_sims) {
#   true_states_hhmm <- numeric(T_sim)
#   true_states_hhmm[1] <- 1
#   for (t in 2:T_sim) {
#     true_states_hhmm[t] <- sample(1:2, size = 1, prob = true_P[true_states_hhmm[t-1], ])
#   }
#
#   simulated_Y_hhmm <- numeric(T_sim)
#   for (t in 1:T_sim) {
#     simulated_Y_hhmm[t] <- rnorm(1, mean = true_mu, sd = true_sigma[true_states_hhmm[t]])
#   }
#
#   fit_sim_hierarchical <- model_hierarchical$sample(
#     data = list(T = T_sim, Y = simulated_Y_hhmm),
#     chains = 4,
#     parallel_chains = 4,
#     adapt_delta = 0.95, # Crucial for hierarchical models
#     refresh = 0,
#     show_messages = FALSE
#   )
#
#   draws_hierarchical <- fit_sim_hierarchical$draws(c("P11", "P22", "sigma_1", "sigma_2"), format = "m
#   cis_hierarchical <- apply(draws_hierarchical, 2, quantile, probs = c(0.05, 0.95))
#
#   coverage_results_hierarchical$P11_captured[i] <- true_P[1,1] >= cis_hierarchical[1, "P11"] && true_
#   coverage_results_hierarchical$P22_captured[i] <- true_P[2,2] >= cis_hierarchical[1, "P22"] && true_
#   coverage_results_hierarchical$sigma1_captured[i] <- true_sigma[1] >= cis_hierarchical[1, "sigma_1"]
#   coverage_results_hierarchical$sigma2_captured[i] <- true_sigma[2] >= cis_hierarchical[1, "sigma_2"]
#
#   if (i %% 10 == 0) cat("Finished simulation", i, "of", n_sims, "\n")
# }
#
# cat("\n--- HIERARCHICAL 100-RUN CALIBRATION RESULTS ---\n")
# print(colMeans(coverage_results_hierarchical))
```

LOO Goodness of fit Heirarchichal

```r
# subset_returns_hhmm <- tail(returns, 300)
# n_total_subset_hhmm <- nrow(subset_returns_hhmm)
# test_weeks_hhmm <- 50
#
# results_list_hierarchical <- list()
#
# cat("Starting 50 rolling LOO predictions for Hierarchical Model...\n")
#
# for (i in test_weeks_hhmm:1) {
#   current_T_hhmm <- n_total_subset_hhmm - i
#   actual_y_hhmm <- subset_returns_hhmm$log_return[current_T_hhmm + 1]
#
#   stan_data_loo_hierarchical <- list(
#     T = current_T_hhmm,
#     Y = subset_returns_hhmm$log_return[1:current_T_hhmm]
#   )
#
#   fit_loop_hierarchical <- model_hierarchical$sample(
#     data = stan_data_loo_hierarchical,
#     chains = 4,
#     parallel_chains = 4,
#     adapt_delta = 0.95,
#     refresh = 0,
#     show_messages = FALSE
#   )
#
#   y_pred_hierarchical <- fit_loop_hierarchical$draws("y_predict", format = "matrix")
#   ci_hierarchical <- quantile(y_pred_hierarchical, probs = c(0.05, 0.95))
#
#   inside_hhmm <- actual_y_hhmm >= ci_hierarchical[1] && actual_y_hhmm <= ci_hierarchical[2]
#
#   results_list_hierarchical[[length(results_list_hierarchical) + 1]] <- data.frame(
#     Date = subset_returns_hhmm$date[current_T_hhmm + 1],
#     Actual = actual_y_hhmm,
#     CI_Lower = ci_hierarchical[1],
#     CI_Upper = ci_hierarchical[2],
#     Captured = inside_hhmm
#   )
#
#   if (i %% 10 == 0) cat("Weeks remaining to test:", i, "\n")
# }
#
# merged_loo_df_hierarchical <- do.call(rbind, results_list_hierarchical)
# proportion_covered_hierarchical <- mean(merged_loo_df_hierarchical$Captured)
#
# cat("\n--- HIERARCHICAL MULTIPLE LOO GOODNESS OF FIT RESULTS ---\n")
# cat("Proportion of true values inside the 90% CI:", proportion_covered_hierarchical, "\n")
```